

Roadmap complet: Debugging Spring Boot pe Linux

Spring Boot · JVM · Server · Linux · Network · Docker · Kubernetes · 24 de săptămâni

Scopul documentului

Acest curriculum te învață să diagnostichezi profesionist aplicații **Spring Boot care rulează pe servere Linux**, de la eroarea HTTP observată de utilizator până la controller, thread pool, JVM, proces, kernel, rețea, storage și servicii dependente.

Ținta nu este memorarea a o sută de comenzi. La final trebuie să poți transforma un simptom ambiguu - „aplicația este lentă”, „serverul a murit”, „primim 502” - într-o investigație sigură, reproductibilă și bazată pe dovezi. Vei ști ce informație să colectezi, în ce ordine, cum să o interpretezi și când să te oprești înainte ca diagnosticul să agraveze incidentul.

Documentul a fost cercetat în august 2026. Recomandările sunt ancorate în documentațiile oficiale Spring, OpenJDK/Oracle JDK, Linux kernel, GNU, systemd, curl, Docker și Kubernetes, listate la final ca [S1]-[S47].

Profilul urmărit

Axă	Ce trebuie demonstrat	Pondere
Spring Boot	Startup, config, HTTP, Actuator, logging	25%
JVM	Threads, CPU, heap, native memory, GC, JFR	25%
Linux	Shell, procese, memorie, disk, systemd	25%
Networking și dependențe	DNS, TCP, TLS, DB, Kafka	15%
Incident response	Triage, dovezi, mitigare, postmortem	10%

Ce vei putea face

La final trebuie să poți demonstra că:

- identifici rapid procesul, versiunea, argumentele JVM, configurația efectivă și porturile;
- diferențiezi o eroare de startup Spring de o problemă a service managerului sau containerului;
- urmărești un request prin proxy, server HTTP, filtre, controller, DB și servicii downstream;
- folosești Actuator fără să expui informații sau operații sensibile;
- citești trei thread dumps succesive și recunoști deadlock, contention, pool exhaustion și loop CPU;
- colectezi și analizezi JFR, heap dump, class histogram, GC logs și Native Memory Tracking;
- diferențiezi Java heap OOM, native OOM, **unable to create native thread** și OOM kill de kernel/cgroup;
- interpretezi load average, CPU states, RSS/VSZ, page faults, swap, I/O wait, PSI și file descriptors;
- diagnostichezi listening socket, DNS, connect timeout, TLS, reset, backlog și connection leak;
- folosești corect quoting, pipes, redirection, exit codes, **find**, **rg**, **awk**, **sed**, **jq** și **xargs**;
- verifici systemd/journald, Docker și Kubernetes fără restarturi oarbe;
- colectezi un evidence bundle și scrii un postmortem cu acțiuni verificabile.

Cum folosești curriculumul

Durată și ritm

Planul are **24 de săptămâni**, la 10-14 ore pe săptămână. Fiecare săptămână conține:

- 2-3 ore de documentație;
- 5-7 ore de laborator;
- 2 ore de injectare controlată a unei defecțiuni;
- 1 oră pentru runbook și explicație orală.

Lucrează într-o mașină virtuală, container sau mediu dedicat. Nu experimenta **kill**, **strace**, heap dumps, firewall sau disk filling pe producție.

Regula de promovare

Pentru fiecare temă trebuie să existe cinci dovezi:

- 1 **Simptom:** poți reproduce problema.
- 2 **Observație:** colectezi semnalul relevant.
- 3 **Ipoteză:** explici mecanismul posibil.
- 4 **Experiment:** falsifici sau confirmi ipoteza.
- 5 **Remediere:** demonstrezi recuperarea și prevenția.

Laboratorul de bază

Pregătește o aplicație Spring Boot cu:

- endpoint rapid, endpoint lent, endpoint care consumă CPU și endpoint care alocă memorie;
- PostgreSQL cu HikariCP;
- un client HTTP către un serviciu mock;
- producer/consumer Kafka opțional;
- Actuator, Micrometer și tracing;
- Docker Compose;
- un unit file systemd într-o VM;
- load generator, de exemplu **hey**, **wrk** sau **k6**;
- fault switches accesibile doar în profilul de laborator.

Nu include endpoint-uri intenționat periculoase în build-ul de producție.

Capitolul 1 - Metoda de debugging

1.1 Debugging versus guessing

Un diagnostic bun micșorează spațiul de căutare. Pentru orice incident notează:

- ce comportament era așteptat și ce s-a observat;
- când a început și ce s-a schimbat;
- impactul: utilizatori, tenants, endpoint-uri, regiuni;
- frecvența și distribuția: toate cererile sau o categorie;
- baseline-ul sănătos;
- ultima versiune/configurație/infrastructură;
- semnalele disponibile înainte să modifichi sistemul.

Nu porni cu „sigur este GC” sau „restartăm”. Pornește cu o cronologie și separă faptele de ipoteze.

1.2 Ordinea investigației

- 1 Confirmă incidentul printr-un SLI apropiat de utilizator.
- 2 Stabilește scope-ul și severitatea.
- 3 Verifică schimbările recente.
- 4 Verifică sănătatea și saturarea resurselor.
- 5 Urmărește request-ul pe straturi.
- 6 Colectează dovezi volatile înainte de restart.
- 7 Mitighează cu cea mai mică schimbare sigură.
- 8 Confirmă recuperarea cu același semnal.
- 9 Determină cauza și factorii contributivi.
- 10 Adaugă prevenție, detecție și test.

1.3 Modelul USE și RED

Pentru resurse folosește **USE**:

- utilization;
- saturation;
- errors.

Pentru servicii folosește **RED**:

- rate;
- errors;
- duration.

Corelează **http.server.requests** cu CPU, GC, thread pools, connection pools și dependențe. O metrică izolată rareori oferă cauza.

1.4 Siguranța în producție

Înainte de o comandă întreabă:

- este read-only?
- poate opri procesul în safepoint?
- produce un fișier mai mare decât spațiul disponibil?
- poate expune secrets, PII sau conținut din memorie?
- crește CPU, I/O sau volumul de logs?
- necesită **sudo**, **ptrace** sau capabilități speciale?
- am timestamp, owner și loc sigur pentru rezultat?

Heap dump-urile pot avea dimensiunea heap-ului și pot conține parole/tokenuri/date personale. **strace**, **tcpdump**, TRACE logging și profileri pot avea overhead și risc de divulgare. Folosește fereastră scurtă, filtre, acces controlat și ștergere conform politicii.

Gate 1

Primești cinci simptome fără cauză. Pentru fiecare scrie primele zece verificări, ce ipoteză testează, costul/risc-ul și criteriul de escaladare. Nu ai voie să propui restart înainte de colectarea dovezilor volatile.

Capitolul 2 - Sintaxa shell necesară

2.1 Anatomia unei comenzi

Shell-ul parsează cuvinte și operatori, aplică expansions, redirections și apoi execută comanda [S20]. Învăță:

- command, options și positional arguments;
- short options (**-f**) și long options (**--follow**);
- **--** pentru a opri interpretarea opțiunilor;

- exit status: **0** succes, non-zero eroare;
- **\$?** pentru ultimul status;
- **command -v, type** și **man**;
- diferența dintre builtin, executable și alias.

Exemple:

```
command -v java
java -version
systemctl status myapp.service --no-pager
echo "$?"
man 1 ps
man 5 proc
```

2.2 Quoting și expansion

- Single quotes păstrează textul literal.
- Double quotes permit expansion, dar previn word splitting și globbing pe rezultat.
- Unquoted variables pot despărți argumente și activa wildcard-uri.
- **\$(command)** face command substitution.
- **\${name:-default}** oferă fallback; **\${name:?message}** cere o valoare.
- *****, **?** și **[]** sunt glob patterns, nu regex.

```
service_name="football-manager.service"
journalctl -u "$service_name" --since "20 minutes ago"
log_dir=${APP_LOG_DIR:-/var/log/myapp}
required_pid=${APP_PID:?APP_PID is required}
```

Nu pune secrets direct în command line: pot apărea în shell history și process listing.

2.3 STDIN, STDOUT, STDERR și redirection

File descriptors standard sunt 0, 1 și 2. Ordinea redirectionărilor contează [S21].

```
command >output.txt
command 2>errors.txt
command >combined.txt 2>&1
command >>append.txt 2>&1
command 2>&1 | tee capture.txt
```

2>&1 >file nu este echivalent cu **>file 2>&1**, deoarece Bash procesează de la stânga la dreapta.

2.4 Pipes, lists și exit codes

Un pipeline conectează output-ul unei comenzi la input-ul următoarei [S22].

```
journalctl -u myapp.service --since today --no-pager | rg 'ERROR|WARN'
ss -lntp | rg ':8080\b'
ps -eo pid,ppid,stat,%cpu,%mem,rss,etime,args --sort=-%cpu | head
```

În scripturile de diagnostic controlate:

```
set -Eeuo pipefail
```

pipefail face pipeline-ul să eșueze dacă o etapă eșuează. **set -e** are reguli subtile; nu îl trata ca substitut pentru verificări și mesaje explicite.

Operatori:

- **a && b**: rulează **b** dacă **a** reușește;
- **a || b**: rulează **b** dacă **a** eșuează;
- **a ; b**: rulează ambele secvențial;
- **cmd &**: rulează în background;
- **()** creează subshell;
- **{ ...; }** grupează în shell-ul curent.

2.5 Variabile, condiții și bucle

```
if systemctl is-active --quiet myapp.service; then
    echo "active"
else
    echo "inactive"
fi

for pid in $(pgrep -f 'myapp.jar'); do
    ps -p "$pid" -o pid,stat,%cpu,%mem,rss,etime,args
done
```

Pentru nume de fișiere necunoscute, evită **for f in \$(find ...)**; newline/spațiile rup parsarea. Folosește delimitare NUL:

```
find /var/log/myapp -type f -name '*.log' -print0 |
    xargs -0 -r ls -lh
```

2.6 Comenzi de text

Stăpânește:

- **rg/grep** pentru căutare;
- **less** pentru navigare;
- **head, tail, cut, sort, uniq, wc**;
- **sed** pentru selecții/substituții simple;
- **awk** pentru coloane și agregări;
- **jq** pentru JSON;
- **tee** pentru vizualizare și salvare;
- **timeout** pentru a limita o comandă.

```
rg -n -C 3 'OutOfMemoryError|deadlock|Connection refused' app.log
awk '{count[$9]++} END {for (code in count) print code, count[code]}' access.log
jq '.components.db.status' health.json
timeout 15s strace -f -p "$pid" -e trace=network
```

Gate 2

Scrie un script read-only care primește service name, colectează status, ultimele logs, proces, sockets și filesystem usage, apoi produce un director timestamped. Trebuie să suporte spații în argumente, erori parțiale și să nu captureze environment/secrets implicit.

Capitolul 3 - Filesystem, permisiuni și pachete

3.1 Navigare și metadata

Comenzi esențiale:

```
pwd
ls -lah /opt/myapp
stat /opt/myapp/app.jar
file /opt/myapp/app.jar
readlink -f /opt/myapp/current
sha256sum /opt/myapp/app.jar
find /opt/myapp -maxdepth 2 -type f -mtime -1 -ls
```

Verifică owner/group, permissions, timestamp, symlink target, checksum și mount. Nu presupune că fișierul pe care îl vezi este artifactul executat.

3.2 Permisii

Înțelege:

- read/write/execute pentru user, group, others;
- efectul execute pe directoare;
- umask;
- ownership cu **chown/chgrp**;
- ACL cu **getfacl**;
- capabilities cu **getcap**;
- SELinux/AppArmor la nivel de diagnostic.

```
namei -l /opt/myapp/config/application.yml
getfacl /opt/myapp/config/application.yml
id myapp
sudo -u myapp test -r /opt/myapp/config/application.yml
```

Nu „repara” cu **chmod 777**. Identifică exact identitatea procesului și permisiunea necesară.

3.3 Disk, inodes și mounts

```
df -hT
df -ih
du -xhd1 /var/log | sort -h
findmnt
lsblk -f
lsof +L1
```

df arată filesystem-ul, **du** însumează fișiere vizibile. Diferențe mari pot apărea din fișiere șterse dar încă deschise; **lsof +L1** le identifică. Un filesystem poate eșua din lipsă de inodes chiar

dacă mai are bytes disponibili.

3.4 Artifact și classpath

```
jar tf app.jar | less
unzip -p app.jar META-INF/MANIFEST.MF
javap -classpath app.jar -p com.example.MyClass
```

Pentru Spring Boot executable jars, verifică **BOOT-INF/classes**, **BOOT-INF/lib**, manifestul, versiunea și checksum-ul. Confirmă JDK-ul efectiv, nu doar cel din shell-ul tău.

Gate 3

Reproduce: config necitibil, symlink greșit, artifact vechi, disk plin, inode exhaustion și log șters dar deschis. Pentru fiecare oferă comandă de confirmare și remediere minimă.

Capitolul 4 - Procese Linux și systemd

4.1 Identificarea procesului

```
pgrep -af 'java.*myapp'
ps -eo pid,ppid,user,stat,lstart,etime,%cpu,%mem,rss,vsz,nlwp,args --sort=-%cpu
pstree -ap
readlink -f /proc/$pid/exe
tr '\0' ' ' </proc/$pid/cmdline
cat /proc/$pid/status
ls -l /proc/$pid/fd | head
```

ps este un snapshot; **top** oferă vedere repetată [S23][S24]. RSS este memoria rezidentă, VSZ este spațiul virtual și nu trebuie interpretat drept memorie fizică folosită. **STAT** ajută: R runnable, S sleeping, D uninterruptible I/O, Z zombie.

4.2 Semnale

Semnale uzuale:

- **SIGTERM**: cere terminare controlată;
- **SIGKILL**: terminare imediată, fără cleanup;
- **SIGQUIT** pentru JVM pe Linux: thread dump în output-ul procesului;
- **SIGHUP**: semantica depinde de aplicație.

```
kill -TERM "$pid"
kill -QUIT "$pid"
```

Nu folosi **kill -9** ca prim pas. Pierzi graceful shutdown și dovezi. Verifică dacă PID-ul mai reprezintă același proces înainte de semnal.

4.3 systemd

```
systemctl status myapp.service --no-pager -l
systemctl show myapp.service
systemctl cat myapp.service
systemctl list-dependencies myapp.service
systemctl show myapp.service -p MainPID,ExecMainStatus,Result,Restart,NRestarts
```

Verifică:

- **ExecStart, WorkingDirectory, User, EnvironmentFile;**
- exit code/signal și restart policy;
- dependency ordering;
- resource limits și cgroup;
- timeout de startup/shutdown;
- versiunea unit file încărcată.

După editarea unit file folosești **systemctl daemon-reload**; restartul serviciului este o acțiune separată și trebuie autorizată.

4.4 journald

journalctl filtrează jurnalul systemd și poate selecta unit, interval, boot și priority [S25].

```
journalctl -u myapp.service -b --no-pager
journalctl -u myapp.service --since '2026-08-04 14:00' --until '2026-08-04 14:20'
journalctl -u myapp.service -p warning..alert
journalctl -u myapp.service -f
journalctl -k -b
journalctl --disk-usage
```

Folosește timestamp-uri cu timezone și păstrează contextul înainte/după eroare.

Gate 4

Construiește un unit file harden-uit pentru laborator. Simulează exit code, SIGTERM lent, WorkingDirectory greșit, EnvironmentFile lipsă și restart loop. Explică diferența dintre aplicație eșuată și unit eșuată.

Capitolul 5 - Startup Spring Boot

5.1 Clasificarea erorilor

Startup-ul poate eșua înainte sau după inițializarea ApplicationContext:

- JDK/artifact/arguments invalide;
- port ocupat sau bind address greșit;
- configurație absentă/incompatibilă;
- profile neașteptate;
- bean creation/circular dependency;
- class/method missing din conflict de versiuni;
- migration sau conexiune DB;
- secret/certificat/keystore;
- health/readiness nereușit după pornire.

Păstrează prima excepție relevantă și lanțul **Caused by**. Ultima linie nu este întotdeauna cauza inițială.

5.2 Debug și condition evaluation

Spring Boot **--debug** activează informații suplimentare pentru o selecție de loggers, nu DEBUG global [S1]. Condition Evaluation Report explică de ce auto-configurările s-au aplicat sau nu.

```
java -jar app.jar --debug
java -jar app.jar --spring.profiles.active=staging
```

În producție, activează loggerul strict necesar și pentru interval scurt. TRACE global poate genera volum masiv și date sensibile.

5.3 Configurația efectivă

Înțelege precedence pentru property sources și verifică:

- command-line arguments;
- system properties și environment variables;
- profile-specific files;
- external config locations;
- Config Server/secret mounts;
- relaxed binding și numele variabilelor;
- placeholders și type conversion.

Nu exporta tot environment-ul într-un ticket. Actuator **env** și **configprops** sunt sensibile și trebuie securizate/sanitizate.

5.4 Dependency și classpath failures

Simptome:

- **ClassNotFoundException;**
- **NoClassDefFoundError;**
- **NoSuchMethodError;**
- **LinkageError;**
- duplicate logging bindings.

Folosește dependency tree în build, inspectează **BOOT-INF/lib**, BOM-ul Spring și versiunea JDK. **NoSuchMethodError** indică adesea compilare față de o versiune și runtime cu alta.

5.5 Port și bind

```
ss -lntp '( sport = :8080 )'
lsof -nP -iTCP:8080 -sTCP:LISTEN
ip addr show
```

Diferențiază bind pe **127.0.0.1**, **0.0.0.0** și IPv6. În container, **localhost** este namespace-ul containerului, nu hostul sau alt container.

Gate 5

Reproduce zece startup failures. Pentru fiecare păstrează logul minim, cauza, comanda de confirmare și fixul. Include port conflict, profile greșit, bean absent, circular dependency, migration failure și versiune incompatibilă.

Capitolul 6 - HTTP, embedded server și request path

6.1 Urmărește request-ul

Traseul tipic:

client - DNS - load balancer/proxy - TLS - socket - Tomcat/Jetty/Netty - filters - Spring Security - DispatcherServlet/WebFlux - controller - service - DB/downstream.

La fiecare hop măsoară:

- connect/TLS/TTFB/total time;
- status și headers;
- request/correlation ID;
- queue și active threads;
- timeout și retry;
- rate/error/duration.

6.2 curl ca instrument

curl oferă timeout separat pentru faza de conectare și pentru transferul total [S28].

```
curl --fail-with-body --silent --show-error \  
  --connect-timeout 2 --max-time 10 \  
  -H 'X-Request-ID: debug-123' \  
  http://127.0.0.1:8080/actuator/health
```

```
curl --verbose --output /dev/null \  
  --write-out 'dns=%{time_namelookup} connect=%{time_connect} '\  
'tls=%{time_appconnect} ttfb=%{time_starttransfer} total=%{time_total}\n' \  
  https://service.example/health
```

```
curl --resolve service.example:443:192.0.2.10 https://service.example/health
```

--resolve separă testul către un IP de rezoluția DNS, păstrând host/SNI. Nu folosi **-k** ca „fix”; doar izolează temporar verificarea certificatului într-un laborator.

6.3 Tomcat și thread pools

Înțelege:

- accept queue, max connections și worker threads;
- keep-alive;
- request/header/body limits;
- connection/read/write timeouts;

- blocking I/O pe worker;
- async processing;
- graceful shutdown.

Spring Boot publică metrics HTTP; metrics Tomcat cer activarea MBean registry conform documentăției [S5]. Corelează **http.server.requests**, active threads, queue, downstream latency și CPU.

6.4 502, 503 și 504

- 502: proxy-ul nu primește un răspuns valid de la upstream.
- 503: serviciul/proxy-ul declară indisponibilitate sau nu are backend eligibil.
- 504: gateway timeout față de upstream.

Verifică unde a fost generat statusul. Un 504 la load balancer nu apare neapărat ca 504 în aplicație; request-ul poate continua după ce clientul a renunțat.

Gate 6

Injectează downstream lent până la saturarea thread pool-ului. Demonstrează succesiunea: latență, coadă, active threads, timeouts, 5xx. Repară prin timeout budget, bulkhead și limitarea concurenței, nu doar prin mărirea pool-ului.

Capitolul 7 - Logging profesionist

7.1 Logs utile

Un eveniment bun conține:

- timestamp cu timezone;
- level;
- service/version/instance;
- thread;
- trace ID/span ID/request ID;
- operație și rezultat;
- durată;
- identificatori de business ne-sensibili;
- exception type și stack trace complet la granița corectă.

Nu loga parole, tokens, cookies, Authorization, chei, PII sau payload-uri brute. Evită aceeași excepție logată la fiecare strat.

7.2 Niveluri și runtime changes

Spring Boot permite configurarea loggerelor și schimbarea lor prin Actuator **loggers** [S3]. Folosește:

- logger pe package/clasă, nu ROOT;

- interval scurt și owner;
- volum observat;
- revenire explicită la nivelul anterior;
- audit pentru schimbare.

```
curl -X POST -H 'Content-Type: application/json' \
  -d '{"configuredLevel":"DEBUG"}' \
  http://127.0.0.1:8081/actuator/loggers/com.example.payment
```

Endpoint-ul trebuie să fie pe management interface securizată. Nu îl expune public.

7.3 Căutare și corelare

```
journalctl -u myapp.service --since '15 min ago' --no-pager |
  rg -n -C 4 'traceId=abc123|ERROR'
```

```
rg -n 'HikariPool.*Timeout|ConnectException|SocketTimeoutException' /var/log/myapp
```

Folosirea **tail -f | grep** este utilă local, dar nu înlocuiește agregarea centralizată, retenția și query-urile după fields.

7.4 Log storms

Un log storm poate consuma CPU, disk, I/O și bugetul platformei. Configurează rotation/retention, limite, sampling pentru evenimente repetitive și alerte pe rate. Nu șterge fișierul deschis; rotește corect sau tratează procesul care îl ține deschis.

Gate 7

Construiește structured logging cu correlation. Reproduce o eroare pe trei servicii și urmărește-o. Activează temporar un logger, măsoară volumul, apoi revino automat. Scrie o politică de redaction.

Capitolul 8 - Actuator, metrics și tracing

8.1 Endpoint-uri

Spring Boot Actuator oferă health, metrics, loggers, threaddump, heapdump și alte endpoint-uri de diagnostic [S4][S6]. Clasifică-le:

- public/minim: de regulă health fără detalii;
- intern read-only: metrics, prometheus, info;
- sensibil: env, configprops, conditions, mappings, beans, threaddump;
- foarte sensibil/greu: heapdump, logfile, loggers write.

Folosește port/adresă de management separată, autentificare, autorizare, network policy și audit. Expune numai ce este necesar.

8.2 Health și probes

Liveness răspunde dacă procesul trebuie repornit; readiness dacă poate primi trafic. Spring avertizează ca liveness să nu depindă de sisteme externe, pentru a evita restarturi în cascadă [S7].

Un DB indisponibil poate face instanța unready, dar nu automat dead. Definește behavior în funcție de capacitatea de degradare.

8.3 Metrics

Spring Boot/Micrometer publică metrics JVM, system, process, disk, HTTP și pool-uri instrumentate [S5]. Urmărește:

- **http.server.requests** rate/error/p95/p99;
- JVM memory/GC/threads/classes;
- process CPU/uptime/file descriptors;
- executor active/queued;
- Hikari active/idle/pending/max;
- Tomcat sessions/threads/connections;
- client HTTP și DB latency;
- Kafka consumer lag și error rate.

Evită high-cardinality tags: user ID, request ID, URL brut sau exception message. Ele pot distruge costul și performanța metric store-ului.

8.4 Tracing

Tracing arată critical path și distribuția latenței. Spring recomandă Micrometer Observation/Tracing pentru integrarea sa, cu suport OpenTelemetry [S8]. Propagă contextul prin HTTP și messaging. Sampling-ul înseamnă că lipsa unui trace nu dovedește lipsa erorii.

Gate 8

Securizează management endpoints, definește health groups și construiește dashboard RED + JVM. Diagnostichează un request lent numai din trace și confirmă cu metrics/logs.

Capitolul 9 - Instrumentele JDK

9.1 Reguli de attach

Oracle recomandă **jcmd** în locul utilităților mai vechi pentru multe diagnostice [S9]. Rulează tool-ul din aceeași versiune JDK, pe aceeași mașină și cu același effective user/group ca JVM-ul [S10][S13]. În container poate fi necesar JDK, PID namespace și permisiune **ptrace**.

```
jcmd -l
jcmd "$pid" help
jcmd "$pid" VM.version
jcmd "$pid" VM.command_line
jcmd "$pid" VM.flags
jcmd "$pid" VM.system_properties
jcmd "$pid" VM.uptime
```

System properties pot conține informații sensibile. Protejează output-ul.

9.2 Thread dump

```
jcmd "$pid" Thread.print -l >threads-1.txt
sleep 10
jcmd "$pid" Thread.print -l >threads-2.txt
sleep 10
jcmd "$pid" Thread.print -l >threads-3.txt
```

O singură fotografie este ambiguă. Trei dump-uri arată progresul. Actuator **threaddump** poate produce JSON [S6], iar **kill -QUIT** scrie dump-ul în output-ul JVM [S9].

9.3 Class histogram și heap dump

```
jcmd "$pid" GC.class_histogram
jcmd "$pid" GC.heap_dump filename=/secure/dumps/heap.hprof
```

Histogramul este mai mic și bun pentru trend, dar nu arată reference paths. Heap dump-ul permite dominator tree și retained size, dar poate cauza pauză/I/O/spațiu și conține date sensibile.

Pregătește preventiv:

```
-XX:+HeapDumpOnOutOfMemoryError
-XX:HeapDumpPath=/secure/dumps
-XX:ErrorFile=/secure/crash/hs_err_pid%p.log
```

Verifică spațiul, permisiunile și retenția înainte de incident [S13].

9.4 Java Flight Recorder

JFR colectează evenimente JVM/aplicație cu overhead mic și este proiectat pentru diagnostic în producție [S12].

```
jcmd "$pid" JFR.start name=incident settings=profile \
  duration=120s filename=/secure/jfr/incident.jfr
jcmd "$pid" JFR.check
jcmd "$pid" JFR.dump name=incident filename=/secure/jfr/snapshot.jfr
jcmd "$pid" JFR.stop name=incident
jfr summary /secure/jfr/incident.jfr
```

Analizează CPU samples, allocation, locks, thread parks, file/socket I/O, GC și safepoints. O înregistrare continuă cu buffer circular păstrează minutele dinaintea incidentului.

9.5 Native Memory Tracking

NMT trebuie activat la startup și urmărește memoria internă HotSpot, nu toate alocările native ale aplicației/JNI.

```
-XX:NativeMemoryTracking=summary

jcmd "$pid" VM.native_memory baseline
jcmd "$pid" VM.native_memory summary.diff scale=MB
```

Gate 9

Pe aceeași problemă colectează metrics, trei thread dumps și JFR. Explică ce informație unică oferă fiecare și costul. Apoi creează un heap dump controlat și documentează chain of custody.

Capitolul 10 - Threads, deadlocks și hangs

10.1 Stările thread-urilor

Interpretează:

- **RUNNABLE**: rulează sau este eligibil; poate fi CPU, syscall sau polling;
- **BLOCKED**: așteaptă monitor lock;
- **WAITING**: așteptare fără timeout;
- **TIMED_WAITING**: sleep/park/wait cu timeout;
- native frames și virtual threads în funcție de JDK.

Nu considera toate WAITING-urile problemă. Workerii idle trebuie să aștepte.

10.2 Pattern-uri

- Același stack RUNNABLE în trei dump-uri + CPU mare: loop/hot method.
- Mulți workers blocați în client DB/HTTP: downstream/pool/timeout.
- Mulți BLOCKED pe același monitor: contention.
- Deadlock section: ciclu de locks.
- Hikari pending + workers așteaptă **getConnection**: pool exhaustion.
- Multe thread-uri nou create: executor necontrolat/leak.
- ForkJoin common pool blocat de I/O: starvation.

Oracle recomandă dump-uri succesive pentru hangs/loops [S11]. Corelează thread names și native thread ID cu **top -H -p PID**.

10.3 Mapping CPU thread

```
top -H -p "$pid"
ps -L -p "$pid" -o pid,tid,psr,stat,pcpu,comm --sort=-pcpu
printf '%x\n' "$tid"
```

Thread dump-urile tradiționale pot afișa **nid** în hex. Convertește TID decimal la hex și găsește stack-ul. Confirmă în mai multe samples sau JFR.

10.4 Thread pools

Pentru fiecare pool cunoaște:

- core/max size;
- queue type/capacity;
- rejection policy;
- active, queued, completed;
- task duration;
- context propagation;
- behavior la shutdown.

Pool size mare poate muta coada în DB și crește memory/context switching. Aplică backpressure și timeouts.

Gate 10

Reproduce deadlock, synchronized contention, infinite loop, blocked downstream și executor queue nebounded. Diagnostichează fără IDE, numai cu Linux, Actuator și JDK tools.

Capitolul 11 - CPU, scheduler și profiling

11.1 CPU nu este același lucru cu load

```
uptime
top
mpstat -P ALL 1 10
pidstat -u -t -p "$pid" 1 10
vmstat -y 1 10
```

Load average include tasks runnable și în uninterruptible sleep; compară cu numărul de CPU-uri și trendul. CPU states relevante: user, system, iowait, steal, idle. În VM/cloud, steal poate indica host contention.

vmstat raportează procese, memorie, paging, block I/O și CPU [S26]. Prima linie este medie de la boot; folosește samples ulterioare.

11.2 CPU throttling și cgroups

În container, aplicația poate vedea hostul dar primi quota limitată. Verifică cgroup v2:

```
cat /proc/$pid/cgroup
cat /sys/fs/cgroup/cpu.max
cat /sys/fs/cgroup/cpu.stat
cat /sys/fs/cgroup/memory.current
cat /sys/fs/cgroup/memory.max
```

nr_throttled și **throttled_usec** în creștere indică limitare CPU. Docker transformă limitele în configurație cgroup [S34].

11.3 perf și JFR

JFR este prima alegere Java-friendly. **perf** poate arăta user/kernel stacks, scheduling și counters, dar necesită symbols, permisiuni și cunoașterea overhead-ului.

```
perf stat -p "$pid" -- sleep 30
perf top -p "$pid"
perf record -F 99 -g -p "$pid" -- sleep 60
```

Nu interpreta sample count ca timp exact fără să înțelegi metoda de sampling, simbolurile și biased/safepoint effects.

11.4 strace

strace urmărește syscalls, signals și schimbări de stare [S29]. Folosește filtre și interval scurt:

```
timeout 15s strace -f -ttT -p "$pid" -e trace=network
timeout 15s strace -f -ttT -p "$pid" -e trace=file
timeout 15s strace -c -f -p "$pid"
```

Poate modifica timing-ul și produce output masiv. Este util când procesul pare blocat în DNS, connect, file I/O, futex sau syscall repetitiv.

Gate 11

Compară busy loop Java, GC pressure, I/O wait și CPU throttling. Pentru fiecare arată pattern-ul în top/pidstat/vmstat, thread dumps și JFR/perf.

Capitolul 12 - Heap, native memory și OOM

12.1 Contabilitatea memoriei

Memoria procesului poate include:

- Java heap;
- Metaspace și compressed class space;
- code cache;
- thread stacks;
- direct/unsafe buffers;
- GC structures;
- memory-mapped files;
- JNI/native libraries;
- allocator fragmentation;
- shared pages.

De aceea **-Xmx** nu este egal cu limita containerului. Lasă headroom pentru native memory și OS.

12.2 Observații Linux

```
free -h
cat /proc/meminfo
cat /proc/$pid/status
cat /proc/$pid/smmaps_rollup
pmap -x "$pid" | tail -1
vmstat -y 1 10
```

MemAvailable este mai util decât **MemFree**; Linux folosește RAM liber ca page cache. RSS nu separă automat toate categoriile JVM.

12.3 Tipuri de OutOfMemoryError

Mesajul contează [S14]:

- **Java heap space**: heap insuficient sau leak;
- **GC overhead limit exceeded**: mult GC, progres mic;
- **Metaspace**: class metadata/leak de classloaders/limită;
- **Direct buffer memory**: direct buffers/limită/cleanup;

- **unable to create native thread:** thread count, stack size, PID/memory/ulimit;
- native allocation failure/out of swap: spațiul nativ/host.

Nu mări **-Xmx** înainte să separi live-set mare legitim de leak. În container, un heap mai mare poate provoca OOM kill mai devreme.

12.4 OOM kill

Un proces poate dispărea fără Java OOM dacă kernelul/cgroup îl omoară:

```
journalctl -k -b | rg -i 'oom|out of memory|killed process'
dmesg -T | rg -i 'oom|killed process'
cat /sys/fs/cgroup/memory.events
systemctl show myapp.service -p Result,ExecMainCode,ExecMainStatus
```

În Kubernetes verifică **lastState.terminated.reason: OOMKilled**, exit code, limits și events.

12.5 Leak workflow

- 1 Confirmă creșterea post-GC, nu doar sawtooth normal.
- 2 Compară class histograms în timp.
- 3 Colectează JFR allocations/live objects.
- 4 Creează heap dump într-un moment reprezentativ.
- 5 Analizează dominators, retained size și paths to GC roots.
- 6 Leagă obiectul de owner/lifecycle din cod.
- 7 Reproduce și testează fixul.

Gate 12

Reproduce Java heap leak, direct buffer pressure, thread leak și cgroup OOM kill. Pentru fiecare identifică sursa corectă de dovezi; un heap dump nu poate explica singur toate cele patru cazuri.

Capitolul 13 - Garbage Collection și safepoints

13.1 Ce măsoară

- allocation rate;
- heap occupancy înainte/după GC;
- pause duration și percentile;
- frequency;
- promotion/old generation pressure;
- concurrent cycle duration;
- humongous allocations unde se aplică;
- CPU consumat de GC;

- safepoint time.

13.2 Unified JVM logging

JDK modern folosește **-Xlog** [S13][S15]. Exemplu controlat:

```
-Xlog:gc*:file=gc.log:time,level,tags:filecount=10,filesize=50M
```

Testează sintaxa pe versiunea JDK țintă. Configurează rotation și asigură-te că directorul este writable.

13.3 Interpretare

- Pause mare rară: obiecte/live set, heap, collector sau host pause.
- GC foarte frecvent cu heap mic după GC: allocation rate mare.
- Heap post-GC crește în timp: posibil leak/live set în creștere.
- Mult CPU GC, progres mic: heap pressure.
- Latență fără GC pause: caută queue, locks, I/O, CPU throttling.

Nu schimba collectorul înainte să ai workload, SLO și baseline. GC tuning nu repară object retention neintenționat.

Gate 13

Capturează GC logs și JFR pentru trei workload-uri: allocation burst, leak și live set legitim mare. Explică diferențele și validează o singură schimbare de configurare prin A/B test.

Capitolul 14 - Network debugging

14.1 Modelul pe straturi

Separă:

- name resolution;
- route/interface;
- TCP connect;
- TLS handshake/validation;
- HTTP request/response;
- application protocol;
- timeout/retry la client.

„Connection refused” înseamnă de regulă că destinația a răspuns fără listener; timeout poate indica routing/firewall/drop/overload. Reset înseamnă terminare abruptă de un endpoint sau intermediar.

14.2 DNS

```
getent ahosts db.example.internal
dig db.example.internal A
dig db.example.internal AAAA
cat /etc/resolv.conf
resolvectl status
```

getent urmează Name Service Switch-ul sistemului și poate reproduce mai bine aplicația decât un query DNS izolat. Verifică TTL, search domains, IPv4/IPv6 și diferența host/container.

14.3 Interfaces, routes și sockets

ip gestionează/afișează interfaces, addresses și routes [S31]; **ss** investighează sockets și stări TCP [S30].

```
ip -br addr
ip route
ip route get 198.51.100.10
ss -s
ss -lnp
ss -tan state established
ss -tan state time-wait
ss -tanp '( sport = :8080 or dport = :5432 )'
lsof -nP -p "$pid" -a -i
```

Urmărește LISTEN address, backlog, SYN-RECV, ESTABLISHED, CLOSE-WAIT și TIME-WAIT. Multe CLOSE-WAIT pot indica faptul că aplicația nu închide sockets după ce peer-ul a închis.

14.4 TLS

```
openssl s_client -connect service.example:443 \
  -servername service.example -showcerts </dev/null

curl -v --connect-timeout 3 --max-time 10 https://service.example/health
```

Verifică SNI, hostname, chain, CA trust, expiry, protocol/cipher și ceasul sistemului. Nu rezolva certificate invalide prin dezactivarea verificării.

14.5 Packet capture

```
timeout 30s tcpdump -i any -nn -s 0 \
  'host 198.51.100.10 and tcp port 5432' -w /secure/captures/db.pcap
```

Captura poate conține credențiale/date dacă traficul nu este criptat și poate avea volum mare. Filtrează, limitează și tratează fișierul ca sensibil. Analizează handshake, retransmissions, resets și timing; nu presupune că „retransmission” este automat cauza.

Gate 14

Reproduce DNS failure, wrong route, refused, connect timeout, TLS hostname mismatch, expired certificate și connection leak. Pentru fiecare identifică stratul fără să modifice toate straturile simultan.

Capitolul 15 - Disk și I/O

15.1 Simptome

- latență mare cu CPU idle/iowait;

- threads în D state;
- fsync/write lent;
- logs blocate;
- DB lentă pe același host;
- disk full/inodes full;
- container writable layer crescut;
- volume/network storage degradat.

15.2 Comenzi

```
iostat -xz 1 10
pidstat -d -p "$pid" 1 10
vmstat -y 1 10
df -hT
df -ih
du -xhd1 /var/lib | sort -h
lsof +L1
```

Corelează throughput, IOPS, await, queue și utilization cu tipul device-ului. **%util** nu are aceeași interpretare simplă pe storage paralel/virtualizat.

15.3 File descriptors

```
cat /proc/$pid/limits
ls /proc/$pid/fd | wc -l
lsof -p "$pid" | awk '{print $5}' | sort | uniq -c | sort -nr
sysctl fs.file-nr
```

Too many open files poate fi limită per-process sau sistem. Găsește tipul descriptorilor care cresc și ownerul. Mărirea limitei doar amână un leak.

Gate 15

Reproduce disk full, inode exhaustion, deleted-open log, I/O latency și FD leak. Construiește dashboard și alerts înainte de pragul de indisponibilitate.

Capitolul 16 - PostgreSQL, HikariCP și tranzacții

16.1 Pool exhaustion

Semnale:

- Hikari pending crește;
- active aproape de max, idle zero;
- acquisition timeout;
- thread dumps în **getConnection**;
- DB sessions/locks/query latency cresc.

Cauze posibile:

- queries lente;
- tranzații lungi;
- connection leak;
- pool prea mic sau DB saturată;
- request concurrency necontrolată;
- timeout mismatch;
- DB/network indisponibil.

Nu mări pool-ul înainte să verifici capacitatea DB. Poți transforma o coadă în sute de query-uri concurente mai lente.

16.2 SQL observability

Folosește logs/metrics selective:

- query duration și rows;
- pool acquisition/usage;
- transaction duration;
- slow query log/**pg_stat_activity**;
- waits și locks;
- **EXPLAIN (ANALYZE, BUFFERS)** în mediu sigur;
- query fingerprint, nu SQL cu PII.

```
select pid, state, wait_event_type, wait_event,
       now() - query_start as query_age,
       left(query, 120) as query
from pg_stat_activity
where datname = current_database()
order by query_start;
```

16.3 Locks și deadlocks

Diferențiază Java monitor deadlock de database deadlock. Urmărește blocking PID, blocked PID, transaction age și query. Kill/cancel de session este acțiune distructivă și cere owner/impact/rollback.

16.4 Hibernate

Diagnostichează:

- N+1 prin query count;
- lazy loading în afara tranzației;
- flush neașteptat;
- fetch join care multiplică rânduri;
- batch absent;
- OSIV care prelungește accesul la DB;

- conexiune ținută în timp ce se face I/O extern.

Gate 16

Reproduce pool exhaustion din query lent, leak și DB outage. Pentru fiecare arată Hikari metrics, thread dumps, **pg_stat_activity** și remedierea corectă.

Capitolul 17 - Kafka și servicii downstream

17.1 Kafka

Pentru consumer verifică:

- lag per partition;
- records consumed rate;
- poll interval și processing time;
- rebalances;
- commit failures;
- deserialization/poison messages;
- retries/DLT;
- broker/network latency.

Thread dump-ul poate arăta consumer blocat în handler, dar lag-ul și partition distribution arată impactul. Nu reseta offsets fără plan și autorizație: poți pierde sau reprocesa date.

17.2 HTTP clients

Configurează separat:

- DNS/connect timeout;
- TLS handshake;
- response/read timeout;
- connection pool acquisition;
- total deadline;
- retry numai pentru operații/erori potrivite;
- circuit breaker și bulkhead.

Un retry fără budget și jitter poate amplifica incidentul. Corelează client metrics cu server metrics și trace.

17.3 Dependency matrix

Pentru fiecare dependență păstrează:

Dependență	Semnal	Fallback
PostgreSQL	pool, query, locks	read-only/degraded dacă se poate

Dependență	Semnal	Fallback
Redis	timeout, hit ratio	source of truth
Kafka	lag, publish errors	outbox/retry
HTTP API	connect/TTFB/status	cache/circuit/fail closed-open
Object storage	request latency/errors	queue/retry

Gate 17

Injectează latență și failure pe fiecare dependency. Demonstrează timeout budget, circuit behavior, lipsa retry storm și revenirea fără restart.

Capitolul 18 - Docker și cgroups

18.1 Identificare

```
docker ps --no-trunc
docker inspect myapp
docker logs --since 20m --timestamps myapp
docker stats --no-stream myapp
docker top myapp
docker exec myapp sh -c 'cat /proc/1/status'
```

Docker logs expune STDOUT/STDERR ale containerului [S35], iar **docker stats** raportează CPU, memorie, network și block I/O [S33].

18.2 Namespace-uri

PID 1 în container poate fi Java. Host PID poate fi diferit. Network, mount și PID namespaces schimbă ce vezi. Confirmă dacă diagnosticul rulează pe host sau în container.

Minimal images pot să nu includă shell/JDK tools. Nu instala ad-hoc în containerul de producție; folosește o imagine de debug compatibilă, sidecar/ephemeral container sau host tools autorizate.

18.3 Limits și OOM

Verifică limits, reservations, cgroup **memory.events**, CPU throttling și PIDs limit. Docker documentează CPU/memory constraints și maparea lor la cgroups [S34]. Corelează heap + native headroom cu limita containerului.

18.4 Image și deployment

```
docker inspect --format '{{.Image}}' myapp
docker image inspect IMAGE_ID
docker history --no-trunc IMAGE_ID
```

Compară digest, labels, environment names fără a afișa secrets, mounts, command și healthcheck. „Aceași tag” nu garantează aceiași bytes dacă tag-ul este mutabil.

Gate 18

Reproduce crash loop, wrong image, missing mount, port nepublicat, CPU throttling și OOM kill. Diagnostichează din host și din namespace-ul containerului.

Capitolul 19 - Kubernetes

19.1 Flux de triage

```
kubectl get pods -n myns -o wide
kubectl describe pod myapp-abc -n myns
kubectl logs myapp-abc -n myns --since=20m --timestamps
kubectl logs myapp-abc -n myns --previous
kubectl get events -n myns --sort-by=.lastTimestamp
kubectl top pod myapp-abc -n myns --containers
```

Documentația Kubernetes recomandă inspectarea Pods, Services, failures și debug containers [S36][S37]. **--previous** este critic după restart.

19.2 Statusuri

- Pending: scheduling, image, volume, quota;
- CrashLoopBackOff: procesul pornește și eșuează repetat;
- ImagePullBackOff: registry/auth/tag/network;
- OOMKilled: memory limit;
- Running dar unready: probe/dependency/config;
- Evicted: node pressure;
- Terminating: finalizer/preStop/grace period/storage.

CrashLoopBackOff este backoff behavior, nu cauza. Cauza este în last state, exit code, logs și events.

19.3 Service și networking

```
kubectl get svc,Endpoints,EndpointSlices -n myns
kubectl describe svc myapp -n myns
kubectl get networkpolicy -n myns
```

Verifică selectors, targetPort, ready endpoints, DNS și network policy. Testează dintr-un pod din același context, nu doar de pe laptop.

19.4 Ephemeral debugging

kubectl debug poate adăuga ephemeral container sau crea debug pod/node session [S38]. Necesită RBAC și poate expune namespace/process/filesystem. Folosește image aprobată și audit.

Gate 19

Reproduce cele șase statusuri. Pentru un 503, urmărește ingress - service - EndpointSlice - readiness - container - Spring health.

Capitolul 20 - Security și debugging

20.1 Remote debugger

JDWP suspendă/inspectează și poate permite control puternic al JVM. Nu expune portul pe internet și nu îl activa implicit în producție. Pentru local/staging izolat:

```
-agentlib:jdwp=transport=dt_socket,server=y,suspend=n,address=127.0.0.1:5005
```

Folosește tunnel controlat, firewall, autentificare la infrastructură și fereastră scurtă. Breakpoints pot opri toate thread-urile și produce outage.

20.2 Least privilege

- nu copia secrets în evidence bundles;
- redactează headers/query/payload;
- limitează Actuator prin rețea și rol;
- protejează heap/JFR/core/pcap;
- folosește **sudo** numai pentru comanda necesară;
- păstrează audit pentru attach, logger changes și captures;
- nu modifica firewall/SELinux ca prim diagnostic.

20.3 Integritatea dovezilor

Înregistrează host, PID, start time, artifact hash, timestamp/timezone, comandă, operator și checksum-ul rezultatului. Stochează criptat, cu retenție și acces minim.

Gate 20

Fă threat model pentru platforma de diagnostic. Clasifică fiecare artifact: logs, thread dump, JFR, heap, core, pcap și environment. Definește owner, acces și retenție.

Capitolul 21 - Playbook-uri pe simptome

21.1 Aplicația nu pornește

- 1 **systemctl status** / container state / pod lastState.
- 2 Exit code și signal.
- 3 Primele și ultimele logs ale tentativei.
- 4 JDK, artifact hash și command line.
- 5 User, working directory și permissions.
- 6 Config/profile/secrets fără dump complet.
- 7 Port/listener.
- 8 DB migration/dependencies.
- 9 Spring failure analysis și primul **Caused by** relevant.
- 10 Reproduce cu aceeași identitate/config într-un mediu sigur.

21.2 Latență mare

- 1 Scope endpoint/tenant/instance și percentile.
- 2 Trace critical path.
- 3 Rate/errors/retries.
- 4 CPU/load/throttling/GC.
- 5 Server threads și queue.
- 6 Hikari/HTTP client pools.
- 7 DB query/waits/locks.
- 8 Network connect/TLS/TTFB.
- 9 Trei thread dumps/JFR.
- 10 Mitigare: shed load, timeout/circuit, rollback, scale justificat.

21.3 CPU 100%

- 1 Host versus cgroup limit.
- 2 PID și hot TID.
- 3 user/system/steal.
- 4 Trei thread dumps și TID mapping.
- 5 JFR CPU samples.
- 6 GC CPU și allocation.
- 7 Traffic/workload change.
- 8 Profiling limitat.

21.4 Proces blocat, CPU mic

- 1 Request/queue saturation.
- 2 Thread dumps succesive.
- 3 Locks/deadlock.
- 4 DB/HTTP pool waits.
- 5 DNS/network/file syscalls cu **strace** limitat.
- 6 I/O wait/D-state.
- 7 Dependency health și timeouts.

21.5 Memorie în creștere

- 1 RSS/cgroup/heap post-GC trend.
- 2 GC și allocation rate.
- 3 Heap versus native.
- 4 Class histogram diff/NMT diff.
- 5 Thread/direct buffer/class count.

- 6 JFR, apoi heap dump dacă este justificat.
- 7 Reproduce și capacity/load comparison.

21.6 502/503/504

- 1 Identifică generatorul statusului.
- 2 Backend endpoints/readiness.
- 3 Listening port/bind.
- 4 Proxy upstream logs și timing.
- 5 Connection pools/backlog/timeouts.
- 6 Dependency latency.
- 7 Deploy/rollout/events.

21.7 Connection refused/timeout/reset

- 1 Resolve host/IP și family.
- 2 Route.
- 3 Listener și bind namespace.
- 4 Firewall/network policy.
- 5 TCP state/capture.
- 6 TLS separat.
- 7 Client timeout și pool.

21.8 Disk full sau read-only filesystem

- 1 **df -hT** și **df -ih**.
- 2 mounts/read-only/kernel logs.
- 3 **du** pe același filesystem.
- 4 deleted-open files.
- 5 logs/dumps/temp/container layer.
- 6 eliberare controlată după identificarea ownerului.
- 7 retenție/alertă/preallocation fix.

21.9 Too many open files

- 1 limits și current FD count.
- 2 FD types/targets.
- 3 trend și traffic.
- 4 sockets states.
- 5 leak de streams/clients/responses.

- 6 fix lifecycle; limit increase numai cu capacity review.

21.10 Kafka lag

- 1 lag per partition și skew.
- 2 consumer state/rebalances.
- 3 processing latency/errors/retries.
- 4 thread pool/DB/downstream.
- 5 max poll și batch settings.
- 6 poison message/DLT.
- 7 scale numai dacă partitions permit.

Gate 21

Rulează un game day cu minimum opt incidente, fără a spune participantului cauza. Scorul se acordă pentru siguranță, timp până la localizarea stratului, dovezi și claritatea handoff-ului, nu pentru viteza restartului.

Capitolul 22 - Evidence bundle și incident response

22.1 Conținut minim

- incident ID, UTC interval și timezone;
- host/pod/container/service;
- PID și process start time;
- application/JDK/OS version și artifact/image digest;
- change/deployment timeline;
- symptoms și SLI graphs;
- logs filtrate;
- process/system snapshots;
- sockets/routes/DNS tests;
- thread dumps/JFR după caz;
- config keys relevante, cu valori sensibile redactate;
- comenzi executate și checksum-uri.

22.2 Snapshot read-only exemplu

```
date --iso-8601=seconds
uname -a
uptime
free -h
df -hT
df -ih
ps -eo pid,ppid,user,stat,%cpu,%mem,rss,nlwp,etime,args --sort=-%cpu
ss -s
systemctl status myapp.service --no-pager -l
journalctl -u myapp.service --since '20 min ago' --no-pager
```

Nu include automat **env**, **/proc/PID/env**, heap, core sau packet capture. Acestea necesită justificare și protecție separată.

22.3 Handoff

Un handoff bun spune:

- impactul acum;
- ce este fapt și ce este ipoteză;
- ce s-a exclus și prin ce dovadă;
- ce mitigare s-a făcut și rezultatul;
- ce risc rămâne;
- următoarea acțiune, owner și deadline;
- linkuri către artifacts și dashboards.

22.4 Postmortem

Include timeline, trigger, root cause, factori contributivi, detection gap, response gap și acțiuni. Fiecare acțiune are owner, termen și verificare. „Să fim mai atenți” nu este acțiune tehnică verificabilă.

Gate 22

Automatizează evidence bundle-ul read-only. Rulează-l în trei medii și demonstrează că nu modifică sistemul, nu expune secrets și continuă dacă o comandă lipsește.

Capitolul 23 - Observabilitate și readiness pentru debugging

23.1 Înainte de incident

Pregătește:

- clocks sincronizate;
- structured logs și correlation;
- metrics RED/USE și saturation;
- tracing cu sampling înțeles;

- version/build/image info;
- secure Actuator;
- JFR continuu;
- GC log rotation;
- heap/crash paths cu spațiu și permisiuni;
- dashboards și alerts;
- runbooks și access pre-approved;
- load/failure tests.

23.2 Alerte

Alerta trebuie să fie legată de impact și să aibă owner/runbook. Exemple:

- error rate și p99 peste SLO;
- readiness instances insuficiente;
- Hikari pending;
- executor queue saturation;
- cgroup memory aproape de limită/OOM events;
- CPU throttling;
- disk/inodes și FD trend;
- Kafka lag age;
- certificate expiry.

23.3 Anti-pattern-uri

- DEBUG global permanent;
- dashboard fără baseline sau unități;
- health care interoghează toate dependențele în liveness;
- retry nelimitat;
- alertă pentru fiecare excepție individuală;
- restart automat care șterge dovezile și maschează leak-ul;
- heap dump configurat pe un disk prea mic;
- instrumente JDK lipsă din strategia de debug a imaginii minimale.

Gate 23

Rulează un readiness review. O persoană nouă trebuie să poată localiza versiunea, dashboards, logs, trace, PID, runbooks și metodele aprobate de colectare în maximum 15 minute.

Plan de studiu pe 24 de săptămâni

Săptămâni	Temă	Livrabil
1	Metodă și siguranță	Triage template + risk matrix
2-3	Bash și text processing	Evidence script sigur
4	Filesystem/permissions	Şase failure labs
5	Procese, signals, systemd	Unit + restart loop lab
6-7	Spring startup/config	Zece startup failures
8	HTTP/Tomcat	Saturation timeline
9	Logging	Correlation + runtime logger runbook
10	Actuator/metrics/traces	Dashboard RED/JVM
11-12	jcmt, dumps și JFR	Diagnostic artifacts
13	Threads/hangs	Cinci thread patterns
14	CPU/perf/strace	Patru CPU/I/O labs
15-16	Memory/GC/OOM	Patru OOM classes
17	Networking/TLS	Şapte network failures
18	Disk/I/O/FD	Cinci resource failures
19	PostgreSQL/Hikari	Pool exhaustion matrix
20	Kafka/downstreams	Retry/timeout game day
21	Docker/cgroups	Limit/crash labs
22	Kubernetes	503 end-to-end trace
23	Security/evidence	Threat model + bundle
24	Capstone	Incident game day + postmortem

Capstone - Incident laboratory

Construieşte un mediu cu două servicii Spring Boot, PostgreSQL, Redis, Kafka și reverse proxy. Rulează pe Linux prin Docker Compose și, opțional, Kubernetes local.

Scenarii obligatorii

- 1 Deployment cu property greșit.
- 2 Port ocupat și bind greșit.
- 3 Downstream lent care saturează workerii.
- 4 Query lock care epuizează Hikari.
- 5 Busy loop pe un endpoint rar.
- 6 Heap leak lent.
- 7 Thread leak și native OOM pressure.
- 8 CPU throttling în container.

- 9 Disk full din log storm.
- 10 FD leak și CLOSE-WAIT.
- 11 TLS certificate/hostname failure.
- 12 Kafka poison message și lag.
- 13 Kubernetes readiness care elimină toate podurile.

Livrabile

- diagramă și dependency inventory;
- SLO și dashboards;
- fault injection scripts;
- evidence bundles;
- thread dumps, JFR și un heap dump securizat;
- câte un runbook per simptom;
- timeline pentru două incidente;
- postmortem;
- demo video fără editarea cauzei dinainte;
- cleanup și cost estimate.

Cheat sheet Linux pentru Spring Boot

Identitate și versiune

```
date --iso-8601=seconds
hostnamectl
uname -a
cat /etc/os-release
java -version
jcmd -l
pgrep -af java
```

Service și logs

```
systemctl status myapp.service --no-pager -l
systemctl show myapp.service -p MainPID,Result,ExecMainStatus,NRestarts
systemctl cat myapp.service
journalctl -u myapp.service -b --no-pager
journalctl -u myapp.service --since '20 min ago' -p warning..alert
```

Proces

```
ps -p "$pid" -o pid,ppid,user,stat,lstart,etime,%cpu,%mem,rss,vsz,nlwp,args
top -H -p "$pid"
cat /proc/$pid/status
cat /proc/$pid/limits
ls /proc/$pid/fd | wc -l
lsof -nP -p "$pid"
```

JVM

```
jcmd "$pid" VM.version
jcmd "$pid" VM.command_line
jcmd "$pid" VM.flags
jcmd "$pid" Thread.print -l
jcmd "$pid" GC.class_histogram
jcmd "$pid" VM.native_memory summary scale=MB
jcmd "$pid" JFR.start name=incident settings=profile \
    duration=120s filename=/secure/incident.jfr
```

CPU și memorie

```
uptime
free -h
vmstat -y 1 10
mpstat -P ALL 1 10
pidstat -u -r -d -p "$pid" 1 10
cat /proc/$pid/smaps_rollup
```

Disk și files

```
df -hT
df -ih
du -xhd1 /var/log | sort -h
findmnt
lsof +L1
iostat -xz 1 10
```

Network

```
ip -br addr
ip route
ss -s
ss -lnpt
ss -tanp '( sport = :8080 or dport = :5432 )'
getent ahosts service.example
curl -v --connect-timeout 2 --max-time 10 https://service.example/health
openssl s_client -connect service.example:443 \
    -servername service.example </dev/null
```

Spring Actuator

```
curl --fail-with-body -sS http://127.0.0.1:8081/actuator/health
curl -sS http://127.0.0.1:8081/actuator/metrics/http.server.requests | jq
curl -sS http://127.0.0.1:8081/actuator/threaddump -o threaddump.json
```

Docker

```
docker ps --no-trunc
docker inspect myapp
docker logs --since 20m --timestamps myapp
docker stats --no-stream myapp
docker top myapp
```

Kubernetes

```
kubectl get pods -n myns -o wide
kubectl describe pod myapp-abc -n myns
kubectl logs myapp-abc -n myns --previous --timestamps
kubectl get events -n myns --sort-by=.lastTimestamp
kubectl top pod myapp-abc -n myns --containers
kubectl get svc,endpointlices -n myns
```

Checklist de competență

Spring și JVM

- [] Clasific startup failures și citesc condition report.
- [] Verific profile/config precedence fără să expun secrets.
- [] Urmăresc request-ul prin server și downstream.
- [] Securizez Actuator și definesc probes corecte.
- [] Folosesc logs/metrics/traces corelat.
- [] Colectez trei thread dumps și identific pattern-uri.
- [] Colectez JFR și îl interpretez.
- [] Separ heap, native, thread și cgroup OOM.
- [] Analizez GC fără tuning orb.

Linux și server

- [] Folosesc quoting, pipes, redirection și exit status corect.
- [] Identific procesul, threads, FDs și artifactul exact.
- [] Diagnostichez systemd și journald.
- [] Interpretez CPU, load, memory, swap și PSI.
- [] Diagnostichez disk, inodes, I/O și deleted-open files.
- [] Separ DNS, route, TCP, TLS și HTTP.
- [] Folosesc strace/perf/tcpdump limitat și sigur.
- [] Înțeleg namespace/cgroup și OOMKilled.
- [] Diagnostichez pod/service/readiness în Kubernetes.

Incident response

- [] Construiesc cronologie și separ fapte de ipoteze.
- [] Colectez dovezi volatile înainte de restart.
- [] Aleg mitigarea minimă și confirm recuperarea.
- [] Protejez artifacts sensibile.
- [] Fac handoff și postmortem verificabil.

Întrebări de interviu

- 1 **Aplicația are CPU 100%. Ce faci?** Scope, cgroup quota, PID/TID, CPU states, thread dumps succesive, JFR, GC și workload.
- 2 **RSS este 3 GB, dar Xmx este 1 GB. De ce?** Native memory, stacks, Metaspace, direct buffers, code cache, mappings și shared pages.

- 3 **Care este diferența dintre Java OOM și OOMKilled?** Primul este aruncat de JVM cu mesaj; al doilea este terminare externă de kernel/cgroup, confirmată în events/kernel/memory.events.
- 4 **De ce trei thread dumps?** Pentru progres și pattern persistent; un singur snapshot poate surprinde stare normală temporară.
- 5 **Ce înseamnă Hikari pending?** Cereri așteaptă conexiuni; cauza poate fi query/tranzacție lentă, leak, DB sau concurență, nu doar pool mic.
- 6 **Cum diagnosticezi 504?** Identifici generatorul, măsori timing per hop, verifici upstream threads/pools/dependencies și timeout budget.
- 7 **Când folosești strace?** Pentru syscalls/signals când suspectezi network/file/futex; filtrat, scurt și cu evaluarea overhead-ului.
- 8 **De ce df și du diferă?** Fișiere șterse dar deschise, mounts și scope; verifici **lssof +L1**.
- 9 **Ce risc are heap dump-ul?** Pauză/I/O/spațiu și conținut sensibil; acces, criptare și retenție.
- 10 **Liveness trebuie să verifice DB?** De regulă nu; dependența externă poate provoca restart cascade. Readiness/degradare se proiectează separat.
- 11 **Connection refused versus timeout?** Refused indică răspuns fără listener; timeout poate fi drop/routing/firewall/overload. Confirmi pe straturi.
- 12 **De ce mărirea thread pool-ului poate agrava?** Mută coada în downstream, crește contention/memorie/context switches și poate produce collapse.
- 13 **Cum corelezi Linux TID cu Java thread?** Găsești hot TID, îl convertești decimal-hex și cauți **nid**, apoi confirmi cu JFR/samples.
- 14 **Ce este CLOSE-WAIT?** Peer-ul a închis, iar procesul local nu a închis socketul; multe pot indica lifecycle leak.
- 15 **Ce colectezi înainte de restart?** Logs, timeline, PID/start, resource snapshots, sockets, thread dumps/JFR și state specific; după risc și timp.

Greșeli frecvente

- restart înainte de dovezi;
- schimbarea simultană a mai multor variabile;
- DEBUG/TRACE global în producție;
- dump de environment cu secrets;
- heap dump pe filesystem aproape plin;
- **kill -9** ca primă reacție;
- **chmod 777** pentru orice permission problem;
- concluzia „memory leak” din RSS fără heap/native/cgroup separation;
- concluzia „CPU problem” din load average singur;
- o singură captură thread dump;
- mărirea Hikari/Tomcat pool fără capacity review;

- retry fără deadline, jitter și idempotency;
- **curl -k** păstrat ca remediare TLS;
- Actuator sensibil expus public;
- liveness legat de toate dependențele;
- ignorarea CPU throttling și memory limit în container;
- folosirea tag-ului Docker ca dovadă de artifact;
- **kubectl logs** fără **--previous** după crash;
- tcpdump/heap/JFR copiate în locații neprotejate;
- comenzi distructive incluse în scriptul automat de evidență.

Bibliografie oficială

Spring Boot și Spring

- [S1] Spring Boot, Logging: <https://docs.spring.io/spring-boot/reference/features/logging.html>
- [S2] Spring Boot, Production-ready Features: <https://docs.spring.io/spring-boot/reference/actuator/>
- [S3] Spring Boot, Loggers: <https://docs.spring.io/spring-boot/reference/actuator/loggers.html>
- [S4] Spring Boot, Actuator REST API: <https://docs.spring.io/spring-boot/api/rest/actuator/>
- [S5] Spring Boot, Metrics: <https://docs.spring.io/spring-boot/reference/actuator/metrics.html>
- [S6] Spring Boot, Thread Dump Endpoint: <https://docs.spring.io/spring-boot/api/rest/actuator/threaddump.html>
- [S7] Spring Boot, Endpoints and Kubernetes Probes: <https://docs.spring.io/spring-boot/reference/actuator/endpoints.html>
- [S8] Spring Boot, Observability: <https://docs.spring.io/spring-boot/reference/actuator/observability.html>

JDK și JVM

- [S9] Oracle JDK, Diagnostic Tools: <https://docs.oracle.com/en/java/javase/25/troubleshoot/diagnostic-tools.html>
- [S10] Oracle JDK, jcmd: <https://docs.oracle.com/en/java/javase/25/docs/specs/man/jcmd.html>
- [S11] Oracle JDK, Troubleshoot Process Hangs and Loops: <https://docs.oracle.com/en/java/javase/25/troubleshoot/troubleshoot-process-hangs-and-loops.html>
- [S12] Oracle JDK, Troubleshoot Performance Issues Using JFR: <https://docs.oracle.com/en/java/javase/25/troubleshoot/troubleshoot-performance-issues-using-jfr.html>
- [S13] Oracle JDK, java Command: <https://docs.oracle.com/en/java/javase/25/docs/specs/man/java.html>
- [S14] Oracle JDK, Troubleshoot Memory Leaks: <https://docs.oracle.com/en/java/javase/25/troubleshoot/troubleshooting-memory-leaks.html>

- [S15] OpenJDK, Unified JVM Logging: <https://openjdk.org/jeps/158>
- [S16] Oracle JDK, Native Memory Tracking: <https://docs.oracle.com/en/java/javase/25/troubleshoot/native-memory-tracking.html>
- [S17] Oracle JDK, jfr Command: <https://docs.oracle.com/en/java/javase/25/docs/specs/man/jfr.html>
- [S18] Oracle JDK, jstack Command: <https://docs.oracle.com/en/java/javase/25/docs/specs/man/jstack.html>
- [S19] Oracle JDK, Troubleshooting Guide: <https://docs.oracle.com/en/java/javase/25/troubleshoot/>

Bash, GNU și Linux

- [S20] GNU Bash Reference Manual: <https://www.gnu.org/software/bash/manual/bash.html>
- [S21] GNU Bash, Redirections: https://www.gnu.org/software/bash/manual/html_node/Redirections.html
- [S22] GNU Bash, Pipelines: https://www.gnu.org/software/bash/manual/html_node/Pipelines.html
- [S23] procps-ng, ps Manual: <https://man7.org/linux/man-pages/man1/ps.1.html>
- [S24] procps-ng, top Manual: <https://man7.org/linux/man-pages/man1/top.1.html>
- [S25] systemd, journalctl: <https://www.freedesktop.org/software/systemd/man/latest/journalctl.html>
- [S26] procps-ng, vmstat Manual: <https://man7.org/linux/man-pages/man8/vmstat.8.html>
- [S27] GNU Coreutils Manual: <https://www.gnu.org/software/coreutils/manual/coreutils.html>
- [S28] curl Command Manual: <https://curl.se/docs/manpage.html>
- [S29] strace Official Documentation: <https://strace.io/>
- [S30] iproute2, ss Manual: <https://man7.org/linux/man-pages/man8/ss.8.html>
- [S31] iproute2, ip Manual: <https://man7.org/linux/man-pages/man8/ip.8.html>
- [S32] Linux Kernel, Control Group v2: <https://docs.kernel.org/admin-guide/cgroup-v2.html>

Containers și orchestration

- [S33] Docker, Container Stats: <https://docs.docker.com/reference/cli/docker/container/stats/>
- [S34] Docker, Resource Constraints: https://docs.docker.com/engine/containers/resource_constraints/
- [S35] Docker, Container Logs: <https://docs.docker.com/engine/logging/>
- [S36] Kubernetes, Troubleshooting Applications: <https://kubernetes.io/docs/tasks/debug/debug-application/>
- [S37] Kubernetes, Debug Running Pods: <https://kubernetes.io/docs/tasks/debug/debug-application/debug-running-pod/>

- [S38] Kubernetes, kubectl debug:
https://kubernetes.io/docs/reference/kubectl/generated/kubectl_debug/
- [S39] Kubernetes, Debug Services:
<https://kubernetes.io/docs/tasks/debug/debug-application/debug-service/>
- [S40] Kubernetes, Pod Lifecycle:
<https://kubernetes.io/docs/concepts/workloads/pods/pod-lifecycle/>

Date, networking și operare

- [S41] PostgreSQL, Monitoring Database Activity:
<https://www.postgresql.org/docs/current/monitoring-stats.html>
- [S42] PostgreSQL, Explicit Locking:
<https://www.postgresql.org/docs/current/explicit-locking.html>
- [S43] Apache Kafka, Monitoring: <https://kafka.apache.org/documentation/#monitoring>
- [S44] systemd, systemctl:
<https://www.freedesktop.org/software/systemd/man/latest/systemctl.html>
- [S45] systemd, coredumpctl:
<https://www.freedesktop.org/software/systemd/man/latest/coredumpctl.html>
- [S46] Linux Kernel, PSI: <https://docs.kernel.org/accounting/psi.html>
- [S47] Linux Kernel, proc Filesystem: <https://docs.kernel.org/filesystems/proc.html>

Ordinea recomandată a practicii

- 1 Stăpânește shell-ul și verificările read-only.
- 2 Reproduce startup și configuration failures.
- 3 Construiește logs, metrics, traces și secure Actuator.
- 4 Învăță thread dumps și JFR înainte de heap tuning.
- 5 Separă CPU, memory, disk și network prin laboratoare.
- 6 Adaugă DB, Kafka și downstream failures.
- 7 Repetă aceleași incidente în systemd, Docker și Kubernetes.
- 8 Rulează game days fără să cunoști cauza.
- 9 Automatizează colectarea sigură și scrie runbook-uri.
- 10 Demonstrează totul prin capstone și postmortem.

Un debugger bun nu este persoana care cunoaște cea mai obscură comandă. Este persoana care păstrează sistemul sigur, colectează semnalul potrivit și reduce metodic incertitudinea până când explicația poate fi demonstrată.